FREE GUIDE • 2026

Master Project Building with AI Tools

WHAT YOU'LL LEARN

A repeatable, phase-by-phase system for building real products with Cursor, Claude Code, Antigravity, Qoder, and similar AI coding tools — without ending up with a tangled, half-working prototype.

Built for PMs, founders, and technical folks who want to ship — not just vibe-code demos.

BY SAKSHAM ARORA

SDE 2 at Intuit • Ex-SDE at Microsoft • 2x Microsoft Intern

topmate.io/sakshamarora • **linkedin.com/in/sakshamarora9575**

Inside this guide

01 Why most AI coding fails

Most people who try to build with Cursor, Claude Code, Antigravity, or Qoder get the same experience: a thrilling first hour, a confusing third hour, and a broken codebase by hour six. The AI seemed to understand. Then it didn't. Then it 'helpfully' refactored something that was working. Then nothing runs.

This is not a tooling problem. The tools are extraordinary. It's a **process problem.** People treat AI coding as a chat with a clever junior engineer, when they should be treating it as **directed construction with an extremely fast, slightly forgetful contractor.**

THE THREE FAILURE PATTERNS

Three failure patterns repeat constantly:

1. No spec. The user and the AI both improvise. Output drifts from intent.

2. No phases. Everything gets built at once, so when something breaks, you can't isolate what.

3. No memory layer. The AI forgets earlier decisions and undoes them. Nothing ever stabilises.

The workflow in this guide directly attacks all three. It's not magic — it's just the discipline that engineering teams already use, adapted for solo builders working with AI tools.

02 The spec-driven mindset

The phrase 'vibe coding' is fun, and it's everywhere. But for anyone trying to build something real — an internal tool, a side project, a startup MVP — vibing is the wrong frame. What actually works is **spec-driven AI development.**

The difference is small in words and huge in outcomes:

Vibe coding	Spec-driven AI building
Prompt and pray	Spec, then prompt
AI decides what gets built	You decide; AI executes
Discover bugs at the end	Test each phase as you go
One giant chat session	Scoped chats, fresh context
Demo-quality output	Ship-quality output

The mental shift is this: **you are the architect, the AI is the construction crew.** An architect doesn't shout instructions while concrete is being poured. They draw plans, review them, then hand them off in stages. Your specs are the plans. The phases are the pours.

THE ONE THING

If you remember nothing else from this guide, remember this: the quality of your specs sets the ceiling on the quality of your output. Every hour spent on the docs folder saves three hours of debugging.

03 The docs folder, in detail

Before you write a single line of code, create a **/docs** folder at the root of your project. This folder is the brain of your build. Every file in it has a job.

The seven files that matter

File	What it does
problemStatement.md	What you're building, who it's for, what success looks like. Written in plain English. No jargon. If you can't explain it here, you can't build it.
architecture.md	The system shape — frontend, backend, database, third-party services, key data flows. Generated by the AI, then reviewed and edited by you.
implementationPlan.md	The phased build plan. Phase 1, Phase 2, Phase 3 — each scoped small enough to test in isolation. This is your roadmap.
conventions.md	Naming conventions, folder structure, error handling patterns, libraries to use, patterns to avoid. The AI re-reads this constantly. It's your highest-leverage file.
edgecases.md	A living document. Start with the obvious cases, add discovered ones as you build. Two sections: 'anticipated' and 'discovered during build'.
evals.md	How you'll verify each phase works. Specific test scenarios, expected behaviour, pass/fail criteria. Written before the phase starts.
decisions.md	Every non-obvious choice and why. 'We chose Zustand over Redux because…' Two lines per decision. Prevents the AI from undoing your earlier reasoning.

TOOLING SHORTCUT

Pro tip: Many AI tools (Cursor, Claude Code, Codex CLI) automatically read files like **CLAUDE.md**, **AGENTS.md**, or **.cursorrules** at the project root. Mirror your *conventions.md* into one of these files so the AI picks it up without being asked. This single trick stops 70% of style drift.

04 The phase-by-phase build loop

This is the heart of the workflow. Every phase follows the same five-step loop. The discipline is in repeating it — even when you're tempted to skip steps because 'this phase is small.'

1. Scope	Open implementationPlan.md. Read only the current phase. Do not let the AI see future phases — they pollute its context and cause premature optimisation.
2. Prompt	Tell the AI to implement *only* this phase, referencing the relevant docs files. Be explicit: 'Read conventions.md and decisions.md before starting.'
3. Review	Don't just accept the diff. Read every file changed. Even a 30-second skim catches 80% of bad patterns before they spread.
4. Test	Run the phase against evals.md. If you can write a quick automated test (Playwright, Vitest, even a Python script), do it. It pays for itself within two phases.
5. Commit	git add, git commit with a clear message. Update decisions.md if anything non-obvious came up. Update edgecases.md with anything new you found.

WHEN TO SPLIT A PHASE

The blast radius rule: if a phase takes more than 90 minutes of AI work without a green test, your phase is too big. Split it. The whole point of phasing is to keep the blast radius of any failure tiny.

05 Context hygiene

Long AI sessions degrade. This is the most under-taught skill in AI coding, and it's the difference between people who ship and people who fight their tool for hours.

Symptoms of a degraded session

- The AI starts hallucinating files or functions that don't exist
- It forgets constraints you set 50 messages ago
- It repeats the same mistake even after correction
- It re-introduces patterns you explicitly rejected earlier
- It confidently produces broken code with confident commit messages

The fix: scoped chats

Treat each phase as its own session. Start fresh. The first message in any new session should be: 'Read these files before doing anything: problemStatement.md, conventions.md, decisions.md, and the section of implementationPlan.md for Phase X.'

This feels wasteful — you 'lose' the chat history. You don't. The history is in your docs folder, where it belongs. The AI re-reads what matters, drops what doesn't, and operates with clean context. This single habit prevents more bugs than any other technique in this guide.

WHEN TO RESET

Rule of thumb: new phase = new chat. Bug-fix session that exceeds 20 turns = new chat. AI starts repeating mistakes = new chat, immediately. Context is cheap to reset and expensive to ignore.

06 Edge cases and evals

Almost every guide tells you to write edge cases upfront. That's correct, but incomplete. Edge cases written upfront are usually the obvious ones — empty input, long input, no internet, wrong format. The brutal edge cases — the ones that break your product in front of a real user — only surface during the build.

Make edgecases.md a living document

Structure it in two clearly-labelled sections:

Section	What goes here
Anticipated	What you and the AI predicted before building. Empty states, errors, large inputs, concurrent users, offline mode, etc.
Discovered during build	Real cases the AI hit during implementation. Weird race conditions, unexpected null values, third-party API quirks. Add to this every time you find one.

evals.md is your acceptance criteria

For every phase, write the eval **before** you start. It should answer: 'How will I know this phase is done?' Concrete scenarios, concrete expected behaviour. If you can't write the eval, you don't understand the phase well enough to build it.

EVAL FORMAT EXAMPLE

Example eval entry for an authentication phase:

Scenario: User signs up with valid email and password

Expected: Redirected to /dashboard, session cookie set, user row in DB with hashed password

Pass criteria: All three observable; no plain-text password anywhere in DB or logs

07 Eval-driven prompting

For tricky logic — auth, payments, data transforms, anything where 'almost right' means 'totally broken' — flip the order. **Write the eval first. Make the AI pass it.**

This is test-driven development adapted for AI tools, and it's wildly effective. Without an eval, the AI optimises for vibes — code that looks right. With an eval, the AI optimises for the only thing that matters: the test passing.

The eval-first prompt pattern

1. Describe the behaviour you want in evals.md. Specific inputs, specific outputs.
2. Ask the AI to write a failing test that captures that behaviour.
3. Run the test. Confirm it fails for the right reason.
4. Now ask the AI to make the test pass — and only that test.
5. Refactor with confidence; the test catches regressions.

WHY EVAL-FIRST WINS

Why this works: AI tools are extraordinary at making a specific test pass. They are mediocre at hitting a vague goal. Your job is to convert vague goals into specific tests. That's it. That's the whole skill.

08 Git rituals that save you

If you're a PM or non-engineer, this section might feel boring. Read it anyway. The single biggest difference between people who recover from a broken AI session and people who lose hours of work is whether they committed regularly.

The phase-completion ritual

Every time a phase passes its evals, run these three commands. Make it muscle memory:

```
git add .
git commit -m "Phase 3: user authentication — passing evals"
git tag phase-3-complete
```

The tag is the magic. If phase 5 breaks something that was working in phase 3, you can *git checkout phase-3-complete* and see exactly what the working version looked like. Without tags, you'll be scrolling through commit history at 1am trying to remember when things last worked.

Branch for risky AI experiments

Before asking the AI to do anything ambitious — a refactor, a tricky integration, a 'try a different approach' — branch first. *git checkout -b experiment-redux-to-zustand*. If the experiment fails, throw the branch away. If it works, merge it. Cheap insurance.

09 The full workflow checklist

Print this. Stick it next to your monitor. Refer to it for every project until it becomes second nature.

Before you start

- ■ Created /docs folder
- ■ Wrote problemStatement.md in plain English
- ■ AI generated architecture.md; you reviewed and edited it
- ■ AI generated implementationPlan.md with clear phases
- ■ Wrote conventions.md (mirrored to CLAUDE.md / AGENTS.md / .cursorrules)
- ■ Drafted edgecases.md with anticipated cases
- ■ Wrote evals.md for at least Phase 1
- ■ Initialised git, made initial commit

For every phase

- ■ Started a fresh chat session
- ■ Pointed the AI at the relevant docs files
- ■ AI implemented only this phase, nothing more
- ■ Reviewed every file the AI changed
- ■ Ran the phase against evals.md
- ■ Wrote at least one automated test (when applicable)
- ■ Updated edgecases.md with anything discovered
- ■ Updated decisions.md with any non-obvious choices
- ■ Committed and tagged the phase

Before you call it done

- ■ Manually tested the full happy path end-to-end
- ■ Manually tested at least three edge cases from edgecases.md
- ■ All evals passing
- ■ README.md written for future-you
- ■ Final commit, final tag

10 Common mistakes & fixes

✗ Treating the AI as a chat partner instead of a contractor

→ **Fix:** Stop having conversations. Give scoped instructions referencing your docs. The AI performs better with structure than with banter.

✗ Letting the AI build features you didn't spec

→ **Fix:** If it's not in implementationPlan.md, it doesn't get built this session. 'Helpful' extras are how scope spirals out and architecture drifts.

✗ Skipping evals because the phase 'looked fine'

→ **Fix:** Looking fine is not working. Five minutes of testing now saves an hour of debugging in phase 5 when you don't know which phase introduced the bug.

✗ Working in one giant chat for hours

→ **Fix:** New phase, new chat. Bug session past 20 turns, new chat. AI repeating mistakes, new chat. Context resets are free. Use them.

✗ Manual testing only at the very end

→ **Fix:** By the end, the cheap-fix window has closed. Test each phase. Even sloppy testing between phases beats thorough testing at the end.

✗ Not tagging your git commits

→ **Fix:** When something breaks, you need to know exactly which working state to roll back to. Tags are your time machine. Use them.

✗ Letting conventions.md go stale

→ **Fix:** If you make a new architectural decision, update conventions.md the same day. The AI only knows what your docs say. Stale docs = drifting code.

That's the whole system.

Spec first. Phase by phase. Test as you go. Reset context often. Commit and tag.

It's not glamorous. It doesn't feel like the future. But it's the difference between shipping a real product and stacking up half-broken prototypes. Every serious AI builder you've ever heard of follows some version of this loop — they just don't always say it out loud.

Run this workflow on your next three projects. By the third one, the discipline becomes invisible. By the fifth, you'll wonder how anyone ever built without it.

Want the full course?

I'm Saksham — currently SDE 2 at Intuit, previously SDE at Microsoft, and a 2x Microsoft intern before that. I teach the complete spec-driven AI development workflow — Cursor, Git, GitHub, AI-powered building across Mac and Windows — on Topmate. Beginner-friendly, hands-on, designed for PMs and technical folks who want to actually ship.

→ **topmate.io/sakshamarora**

Connect on LinkedIn: linkedin.com/in/sakshamarora9575